

Informe algoritmo de generación de números pseudoaleatorios

Estudiantes

Carlos Alberto Barriga Gámez
20222020179

Jose Fernando Ramirez Ortiz
20241020080

Joel David Pérez Arroyave
20242020017

Miguel Ángel Hernández Medina
20222020035

Ciencias de la Computación 2

Agosto de 2026

1. Introducción

Los números pseudoaleatorios son valores generados mediante algoritmos determinísticos que producen secuencias que aparentan ser aleatorias. Estos algoritmos utilizan una semilla como punto inicial, por lo que una misma semilla permite obtener nuevamente la misma secuencia.

En este trabajo se buscó desarrollar un algoritmo para generar números pseudoaleatorios utilizando una semilla y un espacio de 20 valores. Inicialmente se desarrolló un algoritmo propio denominado "GeneradorSimple", tomando como referencia la idea de los generadores congruenciales.

Después de realizar una prueba de frecuencia se observó que los resultados presentaban patrones repetitivos y una distribución muy poco uniforme. Debido a esto, se decidió buscar otro método y se seleccionó Xorshift.

Para comparar los dos algoritmos se generaron 100000 números utilizando un espacio de 20 valores. La frecuencia esperada para cada valor se calculó mediante:

$$F_e = \frac{100000}{20} = 5000 \quad (1)$$

Por lo tanto, cada valor debería aparecer aproximadamente 5000 veces.

2. Desarrollo

2.1. Primer algoritmo: GeneradorSimple

Como primer intento se desarrolló un algoritmo propio llamado **GeneradorSimple**. La idea fue crear un método sencillo, basado parcialmente en la estructura de los generadores congruenciales, utilizando una semilla para obtener los siguientes valores.

En cada iteración se calcula la raíz cúbica de la semilla utilizando `Math.cbrt()`. El resultado se convierte a entero y se utiliza como multiplicador:

```
1 int mult = (int) Math.cbrt(semilla);
```

Después se calcula un incremento utilizando la semilla elevada al valor obtenido anteriormente:

```
1 int incremento = (int) Math.pow(semilla, mult);
```

Con estos valores se obtiene la nueva semilla mediante una operación similar a un generador congruencial:

$$X_{n+1} = (aX_n + c) \text{ mód } 20 \quad (2)$$

En este caso, el multiplicador a y el incremento c no son valores constantes, sino que se calculan a partir de la semilla actual.

El módulo 20 permite limitar los resultados al espacio de valores:

$$0, 1, 2, \dots, 19$$

El código utilizado fue:

```

1 import java.util.Scanner;
2
3 public class GeneradorSimple {
4
5     public static void main(String[] args) {
6
7         Scanner teclado = new Scanner(System.in);
8
9         System.out.print("Ingresa la semilla: ");
10        int semilla = teclado.nextInt();
11
12        int[] frecuencias = new int[20];
13
14        System.out.println(
15            "\nPrimeros 10 n meros generados:");
16
17        for (int i = 0; i < 100000; i++) {
18
19            int mult = (int) Math.cbrt(semilla);
20
21            int incremento =
22                (int) Math.pow(semilla, mult);
23
24            semilla =
25                (mult * semilla + incremento) % 20;
26
27            frecuencias[semilla]++;
28
29            if (i < 10) {
30                System.out.print(semilla + " ");
31            }
32        }
33
34        System.out.println(
35            "\n\n=== VERIFICACI N DE DISTRIBUCI N ===");
36
37        System.out.println(
38            "Cada n mero deber a salir unas 5000 veces.");
39
40        for (int numero = 0; numero < 20; numero++) {
41
42            System.out.println(
43                "N mero " + numero +
44                ": sali " + frecuencias[numero] +
45                " veces");
46        }
47
48        teclado.close();
49    }
50 }

```

2.2. Prueba del GeneradorSimple

Para comprobar la distribución del algoritmo se generaron 100000 números y se contó cuántas veces apareció cada valor.

La primera prueba se realizó utilizando la semilla 20. Los primeros números generados fueron:

1 0 1 2 4 8 0 1 2 4 8

Se puede observar que existe un patrón repetitivo:

$$0 \rightarrow 1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow 0$$

Después de estos cinco valores, la secuencia vuelve a comenzar.

Los resultados de las 100000 generaciones fueron:

Cuadro 1: GeneradorSimple utilizando semilla 20

Valor	Frecuencia
0	20000
1	20000
2	20000
3	0
4	20000
5	0
6	0
7	0
8	20000
9	0
10	0
11	0
12	0
13	0
14	0
15	0
16	0
17	0
18	0
19	0

La frecuencia esperada era de 5000 para cada valor. Sin embargo, los valores 0, 1, 2, 4 y 8 aparecen 20000 veces, mientras que los demás valores no aparecen.

También se realizó una segunda prueba utilizando la semilla 5. Los primeros valores obtenidos fueron:

1 10 0 1 2 4 8 0 1 2 4

Los resultados fueron:

Cuadro 2: GeneradorSimple utilizando semilla 5

Valor	Frecuencia
0	20000
1	20000
2	20000
3	0
4	20000
5	0
6	0
7	0
8	19999
9	0
10	1
11	0
12	0
13	0
14	0
15	0
16	0
17	0
18	0
19	0

Los resultados muestran que el algoritmo no consigue distribuir los 20 valores de manera uniforme. La mayoría de los valores nunca son generados y unos pocos concentran prácticamente todas las generaciones.

Debido a estos resultados, se descartó el **GeneradorSimple** y se buscó otro método de generación.

2.3. Segundo algoritmo: Xorshift

Como alternativa se seleccionó el algoritmo Xorshift. A diferencia del algoritmo anterior, Xorshift utiliza operaciones a nivel de bits para modificar el estado interno del generador.

Las principales operaciones utilizadas son XOR y desplazamientos de bits. En Java, la operación XOR se representa mediante el operador $\wedge$.

El algoritmo utilizado realiza tres transformaciones:

```

1 x ^= (x << 13) & 0xFFFFFFFFL;
2 x ^= (x >>> 17);
3 x ^= (x << 5) & 0xFFFFFFFFL;

```

La primera instrucción desplaza los bits 13 posiciones hacia la izquierda y realiza una operación XOR. La segunda desplaza los bits 17 posiciones hacia la derecha. Finalmente, la tercera desplaza los bits 5 posiciones hacia la izquierda y vuelve a realizar XOR.

El resultado obtenido se guarda nuevamente como estado para generar el siguiente número.

El código utilizado fue:

```

1 import java.util.Locale;
2
3 public class Xorshift {
4
5     long estado;
6
7     public Xorshift(long semilla) {
8         this.estado = semilla & 0xFFFFFFFFL;
9     }
10
11     private long siguienteEntero() {
12
13         long x = estado;
14
15         x ^= (x << 13) & 0xFFFFFFFFL;
16         x ^= (x >>> 17);
17         x ^= (x << 5) & 0xFFFFFFFFL;
18
19         x &= 0xFFFFFFFFL;
20
21         estado = x;
22
23         return x;
24     }
25
26     public double siguienteUniforme() {
27
28         long entero = siguienteEntero();
29
30         return entero / 4294967296.0;
31     }
32
33     public int[] generarEnteros(
34         int cantidad, int min, int max) {
35
36         int[] resultados =
37             new int[cantidad];
38
39         int amplitud =
40             max - min + 1;
41
42         for (int i = 0; i < cantidad; i++) {
43
44             double u = siguienteUniforme();
45
46             resultados[i] =
47                 min + (int)
48                     Math.floor(u * amplitud);
49         }
50
51         return resultados;

```

```

52     }
53
54     public static void main(String[] args) {
55
56         long semilla = 150;
57         int cantidad = 100000;
58
59         Xorshift generador =
60             new Xorshift(semilla);
61
62         int[] numeros =
63             generador.generarEnteros(
64                 cantidad, 1, 20);
65
66         int[] frecuencias = new int[20];
67
68         for (int numero : numeros) {
69             frecuencias[numero - 1]++;
70         }
71
72         double esperada =
73             (double) cantidad / 20;
74
75         System.out.println(
76             "GENERADOR XORSHIFT");
77
78         System.out.println(
79             "Semilla: " + semilla);
80
81         System.out.println(
82             "Cantidad de numeros: " + cantidad);
83
84         System.out.println(
85             "Espacio de valores: 1 - 20");
86
87         System.out.println(
88             "\nFrecuencia esperada: " + esperada);
89
90         System.out.println(
91             "\nValor      Frecuencia      Diferencia");
92
93         for (int i = 0; i < 20; i++) {
94
95             double diferencia =
96                 frecuencias[i] - esperada;
97
98             System.out.printf(
99                 Locale.US,
100                 "%2d %15d %15.0f%n",
101                 i + 1,
102                 frecuencias[i],

```

```

103         diferencia);
104     }
105 }
106 }

```

2.4. Prueba de distribución de Xorshift

Para evaluar Xorshift se utilizó el mismo procedimiento aplicado al primer algoritmo. Se generaron 100000 números dentro del espacio de 1 a 20.

La frecuencia esperada es:

$$F_e = \frac{100000}{20} = 5000 \quad (3)$$

La diferencia entre la frecuencia obtenida y la esperada se calculó mediante:

$$D_i = F_i - F_e \quad (4)$$

Se realizó una primera prueba utilizando la semilla 150.

Cuadro 3: Xorshift utilizando semilla 150

Valor	Frecuencia	Diferencia
1	5042	42
2	5010	10
3	5089	89
4	4873	-127
5	4956	-44
6	4918	-82
7	4880	-120
8	5095	95
9	4973	-27
10	5031	31
11	5104	104
12	4999	-1
13	4982	-18
14	5037	37
15	4969	-31
16	4957	-43
17	4994	-6
18	5073	73
19	5099	99
20	4919	-81

En esta prueba todos los valores aparecen y sus frecuencias se encuentran relativamente cerca de las 5000 apariciones esperadas.

También se realizó una segunda prueba utilizando la semilla 1234.

Cuadro 4: Xorshift utilizando semilla 1234

Valor	Frecuencia	Diferencia
1	5099	99
2	5122	122
3	5030	30
4	4916	-84
5	4921	-79
6	4972	-28
7	5066	66
8	5107	107
9	4899	-101
10	4880	-120
11	5124	124
12	4892	-108
13	4879	-121
14	4887	-113
15	5220	220
16	4980	-20
17	5026	26
18	5044	44
19	4976	-24
20	4960	-40

En esta segunda prueba también se observa un comportamiento aproximadamente uniforme. Aunque existen diferencias respecto a la frecuencia esperada, estas son mucho menores que las observadas en el **GeneradorSimple**.

3. Conclusiones

El desarrollo del presente trabajo comenzó con el algoritmo **GeneradorSimple**, creado como una aproximación al generador congruencial. Este algoritmo utilizaba la raíz cúbica y la potencia de la semilla para calcular los nuevos valores.

Las pruebas realizadas con 100000 números demostraron que este algoritmo generaba ciclos muy cortos. Con las semillas 20 y 5, solamente algunos valores aparecían repetidamente, mientras que la mayoría de los 20 valores posibles nunca eran generados. Debido a que se comprobó que el algoritmo no generaba números bien distribuidos, fue descartado.

Posteriormente se implementó Xorshift, un algoritmo que utiliza operaciones XOR y desplazamientos de bits para modificar el estado interno. Al realizar la misma prueba de frecuencia, las cantidades obtenidas fueron mucho más cercanas a las 5000 apariciones esperadas.

Por lo tanto, para el objetivo planteado, Xorshift presentó un mejor comportamiento que el algoritmo desarrollado inicialmente. La prueba de frecuencia permitió comprobar de manera sencilla que Xorshift produce una distribución aproximadamente uniforme dentro del espacio de 20 valores utilizado.